

《人工智能通识》

第六讲: MCP ——让 AI 动起来的新“万能插座”

周天航

重质油全国重点实验室
中国石油大学 (北京) 未来能源学院

2026 年 9 月



先回顾：上一讲 Function Call 的四步，你还能画出来吗？

- ① 定义工具 用 JSON Schema 告诉模型：有哪些函数、参数是什么
- ② 模型决定 模型不执行，只输出“调用意图”（函数名 + 参数）
- ③ 应用执行 你的程序真的去查天气、查数据库、跑计算
- ④ 结果回填 把结果塞回上下文，模型再组织成自然语言

上一讲留下的一个痛点：每接一个工具，都要写一遍“胶水代码”——工具描述、参数解析、执行、回填；换一个模型、换一个应用，再来一遍。

今天讲的 MCP，目标只有一个：不再重复布线。

情境：你雇了一位世界级助手，可他只能坐在原地



他聪明绝顶，能写代码、分析数据、回答各种问题。可问题是：
他什么都碰不到。

- ▶ 想让他看文件？你得一张张打印出来递到他手里；
- ▶ 想让他查资料？你得亲手把搜索结果拿给他；
- ▶ 想让他用工具？你得先装好、调好，再把结果交给他。

换一份资料重新搬，接一个新工具重新布线——再厉害的助手，也被困在“**信息孤岛**”里。

课堂互动① · 举手投票 (1 分钟)

回想你用豆包 / DeepSeek / Kimi 的经历，它替你“动过手”吗？

A. 只聊过天 B. 联网搜过资料 C. 读过我上传的文件 D. 操作过别的软件 (日历、表格…)

选 D 的同学说一句：是哪个软件？接得顺，还是别扭？

本讲学习目标

知识目标

说清 MCP 解决的问题; 分清 Host / Client / Server 三个角色和 Tools / Resources / Prompts 三类能力; 能按顺序讲出一次 MCP 调用的完整链路。

能力目标

面对一个具体需求, 能判断“写一次 Function Call 就够”还是“值得做成 MCP Server”; 能为实验室的仪器、文献库、SOP 文档设计一份 MCP 能力清单。

素养目标

理解“标准化”对工程生态的价值; 给 AI 接上真实系统时, 先想权限与核对, 再想功能。

全程记住一个对照: MCP 之于 AI 工具, 就像 Modbus / OPC UA 之于现场仪表。

第一部分 · MCP 解决了什么问题？

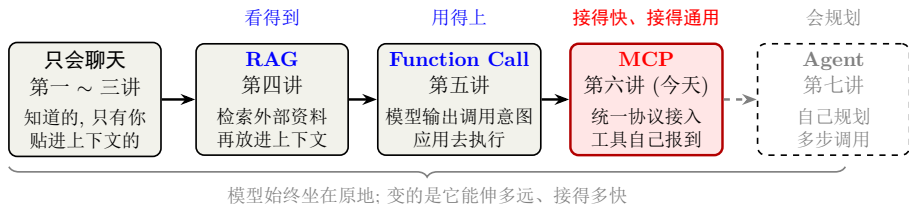
大模型的三重困境

- 够不着** 无法直接访问外部知识，只能依赖你贴进上下文的那点内容
- 不会用** 不能天然调用工具，每接一个新接口都要写一套专门的代码和提示词
- 不通用** 接入方式千差万别，每一次都像“重新造轮子”，费时又难维护

于是，即便有了 **RAG** 帮模型查资料 (第四讲)，有了 **Function Call** 帮模型用工具 (第五讲)，
依然绕不开“**重复接入、重复维护**”的麻烦。

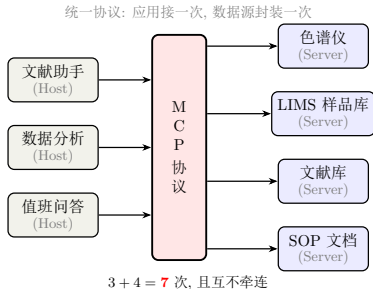
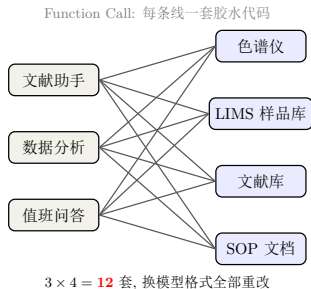
真正缺的，是一个**统一的标准**：让模型能更自然、更安全、更通用地访问各种数据源和工具。

这门课的一条主线：大模型是怎么一步步“伸出手”的



RAG 让模型**看得到**外部资料, Function Call 让模型**用得上**外部工具, MCP 让工具**接得快、接得通用**; 下一讲的 Agent 再让模型**自己规划**多步调用。

算一笔账：胶水代码到底要写多少遍？



实验室要做 3 个 AI 应用 (文献助手 / 数据分析 / 值班问答), 各自要接 4 个数据源 (色谱仪 / LIMS 样品库 / 文献库 / SOP 文档)。

按 Function Call 的做法: 每个应用都要为每个数据源写一遍工具定义、执行、回填—— $3 \times 4 = 12$ 套;
换一家模型 API 的格式, 12 套再改一遍。

有了统一协议: 数据源各封装一次 (4 个 Server), 应用各接一次协议 (3 个 Host)
—— $3 + 4 = 7$ 次, 且互不牵连。

化工同学最熟悉的先例：仪表通信协议的三个阶段

	装置现场	AI 世界的对应
各说各话	每家仪表一套私有协议，每接一台都定制通信卡	每个应用自己写函数、自己接工具
统一报文	Modbus: 报文格式统一了，但“第 40001 号寄存器是什么”还得查手册、手工配	Function Call: 调用格式统一了，但工具定义仍散落在各应用里
自我描述	OPC UA: 设备自己声明“我有哪些量、能做什么”，操作站连上即可浏览	MCP : Server 自己声明能力，Host 连上即可发现、调用

从“手工配点”到“连上即发现”，是自动化行业花了二十年走完的路；AI 工具接入正在快速重走一遍。

MCP 是什么：模型上下文协议 (Model Context Protocol)



模型坐在原地，长出“外部手臂”

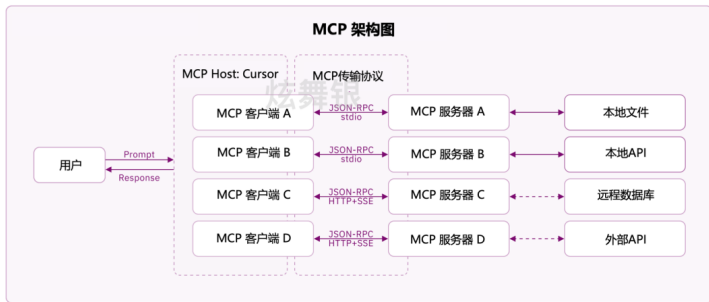
一种**开放标准**，2024 年 11 月由 Anthropic 提出，定义了 AI 模型与外部数据源、工具和环境连接的**统一方式**。

和传统的定制集成不同，MCP 提供了一个**标准化的协议层**，让模型能**自动发现、调用和安全使用**外部能力。

官方给它的类比是“AI 应用的 USB-C 接口”；它已被主流模型厂商和国内多家平台支持，规范与参考实现开源在 github.com/modelcontextprotocol。

支持厂商名单变化很快，以官方仓库为准。

先看一眼全貌: MCP 架构图



Host (这里是编程工具 Cursor) 里跑着四个 Client, 每个 Client 用 **JSON-RPC** 和 **一个** Server 一对一相连;
Server 再去对接真正的资源: 本地文件、本地 API、远程数据库、外部 API。
用户只和 Host 说话。

判断题 (先举手, 再揭晓)

课堂互动②

1. MCP 是一种新的大模型, 比 GPT、DeepSeek 更会用工具。
2. 有了 MCP, 上一讲的 Function Call 就没用了。
3. MCP 是 Anthropic 提出的, 所以只有它家的模型能用。
4. 任何人都可以开发一个 MCP Server, 把自己的服务接进来。

1. ✗ MCP 是协议, 不是模型。它规定的是“工具怎么被发现、怎么被调用”, 模型本身没变。
2. ✗ 模型侧输出“调用意图”仍然靠 Function Call 的机制; MCP 管的是接入侧: 工具在哪定义、怎么被找到。
3. ✗ 开放标准, 任何模型、任何应用都可以实现; 就像 Modbus 不属于某一家仪表厂。
4. ✓ 这正是它的生态意义: Server 由谁写都行, 只要按协议说话。

第一部分小结: 三个关键词

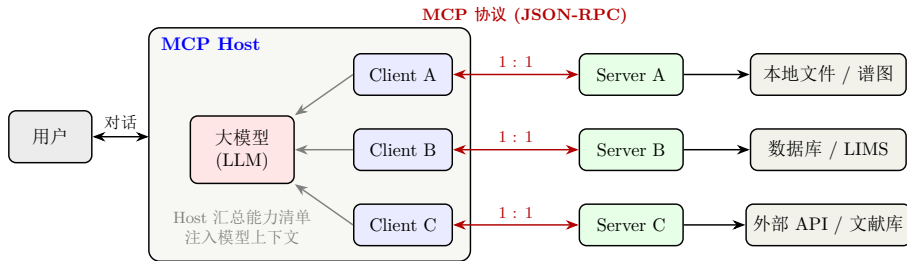
MCP 让大模型**依然坐在原地**, 但能长出“外部手臂”, 根据任务主动伸出去, 抓取所需的上下文与工具, 再用它们完成工作。

- 标准化** 一套协议, 所有工具和数据源都用同一种方式接入
- 自动发现** 连上就知道“对方能干什么”, 不用手工登记
- 一次接入** Server 写一次, 任何支持 MCP 的应用都能用

Q:这个“万能插座”里面到底有哪几个零件? ⇒ 第二部分

第二部分 · MCP 的核心组成: Host / Client / Server / 协议

三个角色, 一种语言



Host = 用户直接使用的应用, 如 Cursor / 值班助手 Server: 把真实能力封装成标准接口 真实世界

Host 是用户直接使用的应用; 它内部为每个 Server 创建一个 **Client**;
Server 把真实世界的能力封装成标准接口; 三者之间说的话, 就是
MCP 协议。

MCP Host ——调度者

Host 就像一个“操作系统环境”，是所有 Client 的**容器与协调中心**。

管理 同时管理多个 MCP Client, 汇总它们各自发现的能力清单

授权 负责认证和授权: 决定当前用户能不能访问某个 Client / Server

兜底 把结果注入模型上下文、呈现给用户; 让整个 Client-Server 调用安全可控

可以这样理解: **编程工具 Cursor 就是一个 MCP Host**; 桌面对话应用、智能体平台、你自己写的 Python 程序, 只要实现了协议, 都可以是 Host。

对照: Host 相当于中控室里的 DCS 操作站——工程师只面对它, 不直接面对现场每一台仪表。

MCP Client —— 会话适配器

Client 是运行在 Host **内部** 的协议适配器, 与某个 Server 建立 **1 : 1** 的标准化连接。

- 发现** 连接建立时, 询问 Server 提供哪些能力 (tools / resources / prompts)
- 翻译** 把 Host / 模型侧的调用请求, 转换成符合协议的标准格式 (JSON-RPC 2.0)
- 回传** 把 Server 返回的结果交给 Host, 由 Host 注入模型上下文
- 维护** 管好这条连接的生命周期: 重连、超时、错误处理

Client **不是** LLM 本身, 用户平时也不会直接接触它——在 Cursor 中**每安装一个 MCP Server, 就会对应生成一个 Client**。

对照: 相当于操作站里为每台仪表加载的一个通信驱动。

MCP Server ——外部能力的接口层

Server 是对**外部服务和资源的适配器**: 把数据库、API、应用、提示模板封装成标准接口。

资源对接 对接真实的外部资源, 把能力描述为一组函数、上下文或模板

动态发现 连接建立时向 Client 声明“我能做什么”(函数列表、参数说明)

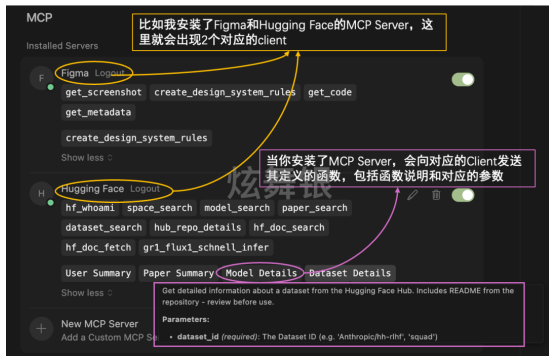
动态调用 收到调用后真正执行操作, 把结果按协议格式返回

安全合规 调外部资源时做鉴权 (API key、OAuth 等), 配合 Host 做权限控制

可以这样理解: Server 就像一个**“工具柜”**, 把真实世界的功能摆好、贴上清晰的标签;

Client 连上柜子, 就能自动发现并取用这些工具。

工具柜长什么样: Cursor 里装了两个 Server

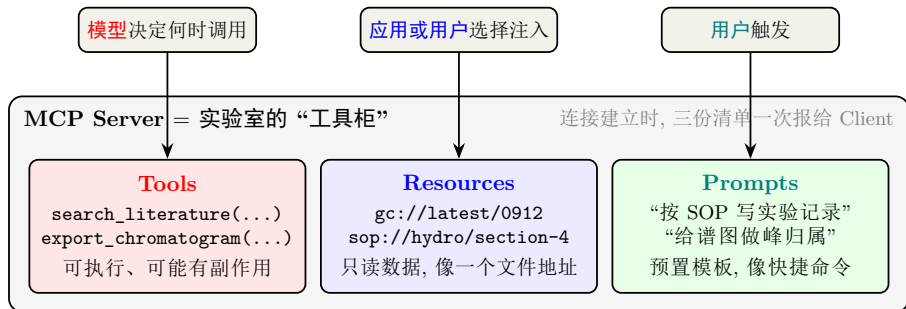


装了 Figma 和 Hugging Face 两个 Server, Host 里就出现两个对应的 Client。

Hugging Face 的 Server 声明了一排工具: `model_search`、`paper_search`、`dataset_search`、`hf_doc_fetch` ...

点开任意一个, 能看到它的说明和参数 (如 `dataset_id`) ——这正是上一讲工具描述的那套 JSON Schema, 只是不再由你手写, 而是 Server 自己报上来的。

Server 能声明的三类能力: Tools / Resources / Prompts



Tools	模型可调用的函数, 可能有副作用	由模型决定何时调用
Resources	只读的数据: 文件、记录、实时读数	由应用或用户选择注入上下文
Prompts	预置的提示模板 / workflow	由用户触发, 像一个快捷命令

MCP 协议——Client 与 Server 的对话语言

底层格式	JSON-RPC 2.0: 消息都是 JSON 结构, 人能读, 程序也好解析
传输通道	<code>stdio</code> : 本地进程间通信, 轻量、无网络依赖; <code>HTTP</code> + <code>SSE</code> : 远程通信, 调云端 Server
消息类型	<code>request</code> (Client 请 Server 做事) / <code>response</code> (Server 返回结果) / <code>notification</code> (无需回应的事件, 如 Server 主动推送“工具列表变了”)
会话机制	先 <code>initialize</code> 握手, 再能力发现 (<code>tools/list</code> 等), 然后才进入具体调用 (<code>tools/call</code>)

如果 Server 是“工具柜”, Client 是“取工具的人”, 那么协议就是“**说话的语言**”和“**传递指令的方式**”。

新版规范已把远程通道改为 Streamable HTTP, 思路不变; 细节以 GitHub 官方仓库为准。

一条消息长什么样？拆开一次 tools/call

Client → Server: request

```
{"jsonrpc": "2.0", "id": 7,  
  "method": "tools/call",  
  "params": {  
    "name": "model_search",  
    "arguments": {"query": "image  
classification",  
                  "limit": 5}}}
```

Server → Client: response

```
{"jsonrpc": "2.0", "id": 7,  
  "result": {  
    "content": [{"type": "text",  
                 "text": "1. google/vit-base  
...\n2. ..."}]},  
  "isError": false}}
```

- ▶ id 把请求和响应配成对——多个调用可以同时在路上;
- ▶ name + arguments 就是上一讲模型输出的“调用意图”，只是被 Client 包成了标准信封;
- ▶ 结果是一段文本 (或图片 / 资源), 会被 Host 塞回模型上下文——和 Function Call 的“结果回填”一模一样。

示例为简化写法, 字段以官方规范为准。

把四个零件搬进中控室：一张对照表

MCP	装置对应物	为什么像
Host	DCS 操作站画面	用户只面对它；汇总所有设备，管权限
Client	操作站里的一个通信驱动	一个驱动对一台设备 (1 : 1), 用户看不见
Server	说 Modbus / OPC UA 的现场仪表	自己声明有哪些点、能做什么
协议	Modbus / OPC UA 报文	统一格式, 谁家的设备都能接
Tools	可写操作: 改设定值、启停泵	有副作用, 要有权限
Resources	只读测量值: 温度、压力、液位	随时读, 不改变现场
Prompts	标准操作票 / 巡检模板	人触发, 按模板走

Q:一台只有 4~20 mA 模拟信号的老仪表, 要靠一个网关才能接进 OPC UA ——对应到 MCP, 这个“网关”是谁?

小组：给本实验室设计一个 MCP Server

课堂互动③

实验室有：一台气相色谱仪（每天产出谱图数据）、一个 2000 篇 PDF 的文献库、一套精馏 / 加氢实验的 SOP 文档。

4 人一组，5 分钟，在草稿纸上写出这个 Server 的能力清单：Tools、Resources、Prompts **各至少 2 条**，每条给出名字和一句话说明。

示例开头：Tools: `search_literature`(关键词, 年份) ——在文献库里检索并返回 5 段最相关的原文…

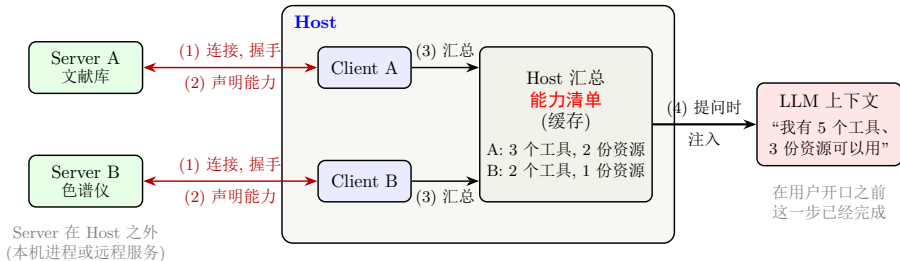
判断标准：

- ▶ 有副作用的放进 Tools，只读的放进 Resources ——“导出谱图”和“读取最新谱图”该各在哪一类？
- ▶ Prompts 是不是真的能省下同学反复敲的那几百字（如“按 SOP 格式写实验记录”）？

选 2 组上黑板写清单，其他组找“放错类”的条目——这份清单第五部分还会用到。

第三部分：一次 MCP 调用的完整链路

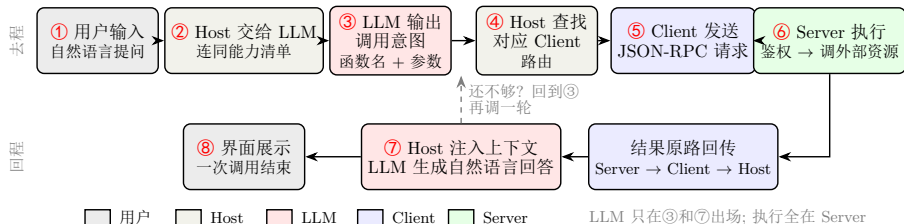
先看“安装时”发生了什么：能力发现



在 Host 里安装一个 Server 时: Host 为它创建一个 Client → 两者握手 → Server 声明自己的函数、资源和 Prompt 模板 → Client 把清单交给 Host → Host **汇总多个 Client 的清单**, 在合适的时机注入模型上下文。

从此模型“知道”自己有哪些手臂可以伸——这一步在提问之前就完成了。

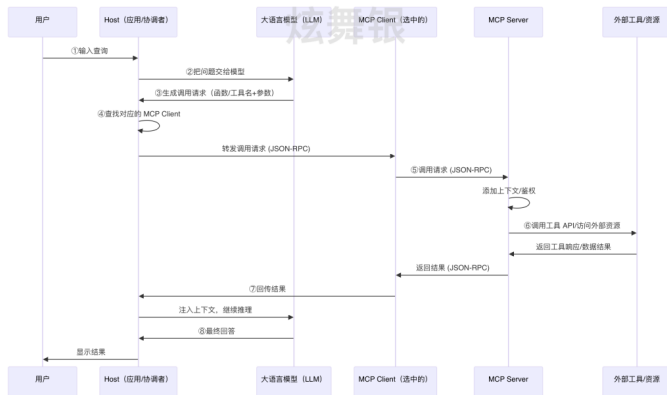
再看“提问时”的八步链路



上面一行是去程 (用户 → 模型 → Client → Server → 外部资源), 下面一行是回程, 原路返回。

注意 LLM 只出现在第③步和第⑦步: 它**决定调什么**和**组织最终回答**, 中间的执行全部不归它管。

同一条链路, 换成时序图看



六个泳道: 用户 / Host / LLM / Client / Server / 外部工具。竖着看时间, 横着看谁在和谁说话。

JSON-RPC 只出现在 Client 与 Server 之间——用户、Host 和 LLM 之间说的都是各自应用内部的话。

去程四步：从一句话到一个调用请求

	谁做	做什么
①	用户	在 Host 界面输入自然语言，如“帮我找一个图像分类的预训练模型”
②	Host	把问题连同已汇总的能力清单交给 LLM 推理
③	LLM	判断需要外部能力，输出结构化的调用意图：函数名 <code>model_search</code> + 参数
④	Host	在缓存的能力清单里查：哪个 Client 暴露了这个工具？找到后把请求路由过去

第③步就是上一讲的 Function Call：模型仍然只“说”不“做”。

第④步是 MCP 新增的：因为工具可能来自多个 Server，Host 得先知道该找谁。

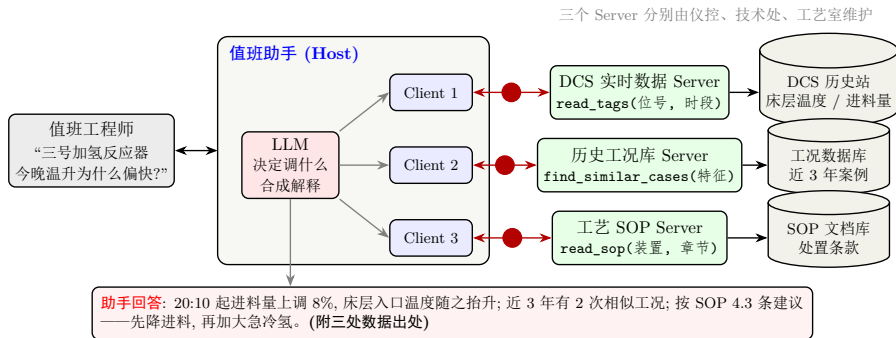
回程四步：从执行结果到最终回答

	谁做	做什么
⑤	Client	把请求包成 JSON-RPC 2.0 消息, 通过自己那条 1 : 1 连接发给 Server
⑥	Server	校验权限 → 调用真正的外部 API / 数据库 / 工具 → 把结果返回给 Client
⑦	Host + LLM	结果 Server → Client → Host 层层回传; Host 把它注入上下文, LLM 据此生成自然语言回答
⑧	Host	把回答显示在界面上; 一次完整的 MCP 调用结束

如果模型觉得还不够 (比如还想查一下数据集), 第③ ~ ⑦步会再循环一轮 ——

这就是下一讲 Agent 的雏形: 多次调用、逐步逼近目标。

把链路搬进炼厂：“三号加氢反应器今晚温升为什么偏快？”



值班助手 (Host) 装了三个 Server: DCS 实时数据、历史工况库、工艺 SOP 文档。

模型先调 read_tags 取近 4 小时的床层温度和进料量,再调

`find_similar_cases` 找相似工况, 最后读 SOP 里的处置条款,

把三份结果合成一段有依据的解释——三个 Server 由三家单位分别维护，助手一次都没改代码。

排一排 + 现场演示

课堂互动④

排一排 (3 分钟): 屏幕上打乱给出八步链路的八张“卡片”(用户输入 / Host 交给 LLM / LLM 输出调用意图 / Host 查找 Client / Client 发 JSON-RPC / Server 执行 / 结果注入上下文 / 界面展示)。同桌两人把顺序写在纸上, 再互换检查: 哪一步最容易放错?

现场演示 (3 分钟): 打开 [阿里云百炼文档](#), 找到 MCP 相关章节, 一起看一个现成 Server 的工具清单: 每个工具的名字、说明、参数各是什么。

看演示时回答:

- ▶ 这份清单是谁写的? 是平台手工登记的, 还是 Server 自己声明的?
- ▶ 如果把这个 Server 换到另一个 Host 里, 清单要重写吗?

平台界面随版本变化, 以现场看到的为准; 重点看“工具清单从哪来”, 不用记具体按钮。

第三部分小结：一次调用，三个要点

- 发现在前** 安装时就完成能力发现，模型提问前已经“知道自己有哪些手”
- 模型只决定** LLM 只在第③步（调什么）和第⑦步（怎么答）出场，执行全部交给 Server
- Host 做路由** 多个 Server 并存时，由 Host 查清单、找 Client、汇总结果

Server 声明能力 → Client 发现能力 → Host 汇总能力并注入
上下文
→ LLM 基于这些能力**自主决定**调用哪个工具。

Q:既然模型侧还是 Function Call, 那 MCP 到底新在哪? ⇒ 第四部分

第四部分 · MCP vs. Function Call: 到底有什么不同?

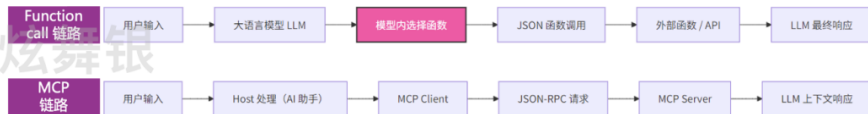
表面上看, 两条链路几乎一样

- 1 用户提出问题
- 2 LLM 决定要不要调用外部工具
- 3 工具执行后返回结果
- 4 LLM 根据结果生成最终回答

很多人因此会问: 不都是“让大模型调用外部函数”吗?

核心差异只有一句话: **函数的定义写在哪里、由谁来发现。**

两条链路并排放: 多出来的是 Client 与 Server



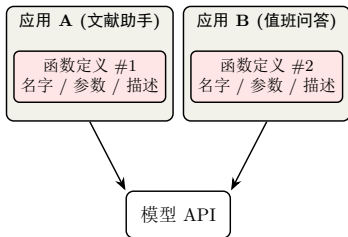
上面一行 Function Call: 用户输入 → 模型 → **模型内选函数** → JSON 参数 → 外部函数 / API → 模型重新回答。

下面一行 MCP: 用户输入 → Host 处理 → **MCP Client** → JSON-RPC 请求 → **MCP Server** → LLM 上下文构建。

多出来的两个环节, 就是把“函数定义”从应用里搬出去的地方。

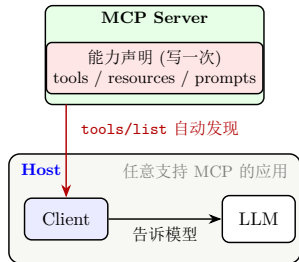
函数定义在哪里：应用层 vs. Server 侧

Function Call: 定义写在应用层



同一个工具，两个应用各注册一遍，每次请求都塞进去；
换一种模型格式，两份都要改

MCP: 定义在 Server 侧，写一次



换一个 Host、换一个模型，Server 不用动；
新 Server 插上就被发现

Function Call: 名字、参数、描述写在应用层，每个应用自己注册一遍——像每家自建插座，规格各不相同。

MCP: 能力声明在 Server 侧，Client 自动发现再告诉 LLM ——万能插座，任何支持 MCP 的 Host 随插随用。

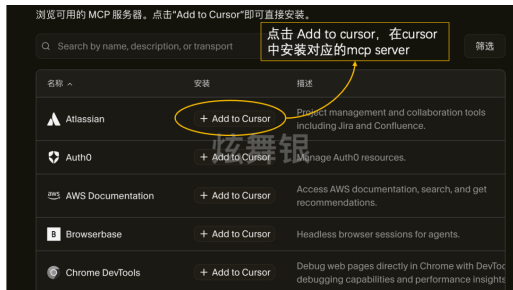
一张对比表

	Function Call	MCP
函数定义在哪	应用层, 每个应用各写一份	Server 侧, 写一次
模型怎么知道	应用把定义手工塞进请求	Client 自动发现, Host 汇总后注入
换应用 / 换模型	重新注册、重新写胶水	接入一次协议即可复用
安全策略	应用层自行实现, 各管各的	协议层统一机制: 鉴权、授权、上下文隔离
生态	各自造轮子, 定义散落各处	谁都能写 Server, 谁都能接
本质	模型侧的一种能力	接入侧的一套协议

两者**不是替代关系**: MCP 链路第③步用的仍是模型的 Function Call 能力

(国内模型的使用法见 [DeepSeek API 文档](#) Function Calling 章节)。

实际案例：一键接入 Hugging Face 的工具



Cursor 里有一份可浏览的 Server 列表: Atlassian、AWS 文档、浏览器、Chrome 调试工具…

点一下 [Add to Cursor](#), Server 就装好了, 对应的 Client 也随之生成。

装上 Hugging Face 的 Server 后, 它会自动声明 `model_search`、`dataset_search` 等工具; Host 把这些信息告诉 LLM。

你没有写过一行函数定义, 也没有注册过任何 API。

国内访问 Hugging Face 资源可用镜像站 hf-mirror.com。

输入一句话，看它自己去调工具

对话框里输入：

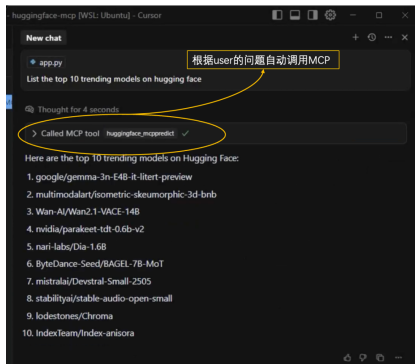
List the top 10 trending
models on hugging face

模型思考几秒，界面显示 **Called MCP tool: huggingface_mcp**，然后直接列出 10 个当前最热门的模式。

对照八步链路：你只做了第①步，看到了第⑧步；中间六步 Host、Client、Server 自己走完。

从“手动布线”到“即插即用”——这就是 MCP 带来的体验差异。

截图来自网络公开的 Cursor + Hugging Face MCP 演示，仅供学习参考。



图源：Cursor + Hugging Face MCP 演示截图（网络公开资料，仅供学习参考）

案例分析：什么时候值得做成 MCP Server？

课堂互动⑤

4 人一组，4 分钟，对下面三个需求分别判断：直接写 Function Call 就够，还是值得封装成 MCP Server？说出理由。

- A.** 你自己的毕业设计脚本，只需要让模型调一次“查物性数据库”。
- B.** 实验室的色谱仪数据，文献助手、数据分析、值班问答三个应用都要读，明年还可能换模型供应商。
- C.** 学院想把教务系统的课表查询开放给多个 AI 助手（豆包、元宝、自研的都可能接）。

判断线索：

- ▶ 有几个应用要接？有几个数据源？乘起来的数字大不大？
- ▶ 谁来维护？数据源的主人愿不愿意“自己声明能力”？
- ▶ 换模型、换应用的可能性有多大？

参考：A 用 Function Call 即可；B、C 都值得做 Server ——但 C 还要先回答“谁能调、能调什么”。

第五部分 · 生态意义与理性看待

AI 的“插件商店”

标准化设计, 让生态有了全新的可能:

任何开发者 都可以实现一个 MCP Server, 把自己的服务 (搜索、数据库、企业内部系统) 包装成标准接口;

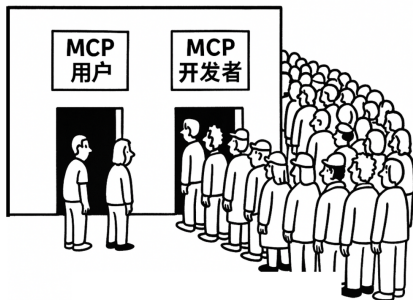
任何 Host 只要支持 MCP (编程工具、桌面助手、客服系统、智能体平台), 就能直接接入这些服务;

对 LLM 来说 看到的是统一的“能力清单”, 而不是一堆零散的私有 API。

一个协议, 连接无数工具, 形成开放的生态网络。

官方仓库 github.com/modelcontextprotocol 下有规范、各语言 SDK 和一批参考 Server (文件系统、数据库、Git …);
国内平台的接法见 [阿里云百炼文档](#) 的 MCP 章节。

但也要看到：它还在早期



MCP 现状：可能是一个开发者比用户还多的技术。

有人打趣说, MCP 的现状是:“**开发者比用户还多**”。

更多是工程师和爱好者在积极构建 Server，普通用户的感知还不强——这恰恰说明它仍处于**早期建设期**。

对你意味着什么:

- ▶ 现在学会看懂“能力清单”，比记住某个按钮更值钱；
- ▶ 规范细节（传输方式、字段名）还在变，别背，会查；
- ▶ 化工行业的 Server（物性库、仪器、DCS 网关）几乎还是空白——机会。

插座越通用, 越要想清楚“插什么、谁能插”

新风险	发生了什么	化工里的对应
上下文膨胀	每个工具的描述都要占 token (第二讲); 装 20 个 Server, 提问前上下文可能已经几千 token	报警点配太多, 真正的报警被淹没
结果里夹带指令	Server 返回的网页 / 文档里可能藏着“忽略之前的要求...”, 模型可能照做	来路不明的 U 盘不能插进 DCS
权限过大	Server 能做什么, 模型就可能做什么; 一个 <code>write_tag</code> 工具就能改设定值	手动阀的钥匙, 不会交给还没上岗的新人

工具越好接, 接之前的那一步判断就越重要——**便利不等于该做。**

负责任地接入：给 AI 一只手，也要给它一道联锁

- ▶ **最小权限**：能只读就不给写；实验数据、DCS 位号默认只暴露 Resources，写操作单独审批；
- ▶ **危险动作要人确认**：改设定值、删除记录、发送邮件这类工具，Host 必须弹出确认，不能全自动；
- ▶ **核对再引用**：模型从 Server 拿回的数字，写进报告前对照原始数据源看一眼，出处要能点开；
- ▶ **数据不乱接**：未公开的实验数据、论文草稿，不接入来路不明的第三方 Server；先看它把数据发到哪；
- ▶ **诚实署名**：用 AI 助手调工具生成的分析，在作业和论文里说明工具与数据来源。

Q:讨论 (2 分钟): 互动③里你们组设计的 Server, 哪一条能力如果被误调用, 后果最严重? 该怎么加“联锁”?

小组讨论: 给实验室 MCP Server 定权限等级

课堂互动⑥

回到互动③的能力清单, 4 人一组, 4 分钟, 把每条能力填进三档:

一档·自动放行: 只读、无副作用, 如 `read_latest_chromatogram`、`search_literature`;

二档·需人确认: 有副作用但可撤销, 如 `export_report`、`append_lab_note`;

三档·禁止暴露: 不可逆或涉及安全, 如 `delete_dataset`、`set_heater_temperature`。

然后回答:

1. 三档的分界线是什么? “能不能撤销”还是“会不会伤人”?
2. 三档里的能力, 如果确实需要 AI 参与, 换一种更安全的暴露方式是什么? (提示: 只给“建议值”, 不给“执行权”)

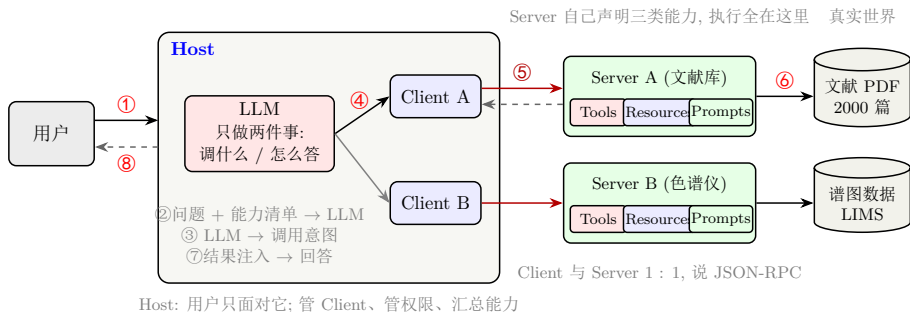
每组交一张表, 选一组讲: 说得出“为什么放二档不放一档”, 就是学会了。

MCP 和前几讲的关系：各管一段

讲次	它管什么	和 MCP 的关系
第四讲 RAG	让模型查到外部资料	一个检索器可以封装成一个 MCP Server
第五讲 Function Call	模型输出调用意图	MCP 链路第③步仍然靠它
第六讲 MCP	工具怎么被发现、接入	把定义从应用层下放到 Server
第七讲 Agent	模型自己规划多步调用	MCP 是它的“手”，Agent 是“脑”
第八讲上下文工程	上下文里放什么	工具描述占 token, 要精选

一句话: RAG 给资料, Function Call 给动作, MCP 给接口, Agent 给规划。

本讲总结: 一个插座, 三个角色, 三类能力, 八步链路



用户只面对 Host; Host 为每个 Server 配一个 Client; Server 声明 Tools / Resources / Prompts;
LLM 只在“决定调什么”和“组织回答”两处出场, 执行全在 Server
——一切靠 JSON-RPC 这种语言。

七个词, 各用一句话带走

MCP	开放协议, 统一模型与外部工具 / 数据的接法, 不是模型
Host	用户直接用的应用, 管多个 Client, 管权限, 汇总能力
Client	Host 内部的适配器, 与一个 Server 1 : 1 相连
Server	工具柜: 把真实资源封装成标准接口, 自己声明能力
Tools	模型可调用的函数, 可能有副作用
Resources	只读数据, 由应用或用户选择注入
Prompts	预置模板, 用户触发

Q:回到开场: 那位“困在原地的助手”, 现在你能说清他是怎么长出手臂、又该拿多大权限的了吗?

随堂快测

课堂互动 · 3 道题

- Q1 (单选) 在 Cursor 里安装了 3 个 MCP Server, Host 内部会有几个 MCP Client?
A. 1 个 B. 3 个 C. 由模型决定 D. 0 个, Client 在 Server 那边
- Q2 (判断) MCP 与 Function Call 的核心区别在于: MCP 把函数定义从应用层下放到了 Server 侧。
- Q3 (简答) 实验室想让值班助手读 DCS 数据。“读取温度位号”和“修改温度设定值”分别应该做成哪类能力? 各给一个理由。

答案: Q1 选 **B**, Client 与 Server 1 : 1; Q2 **✓** 正确, 模型侧的调用意图机制并没有变;

Q3 读位号做成 Resources (只读、无副作用, 可自动放行); 改设定值若非做不可, 只能做成需人确认的 Tool, 或只输出建议值。

线上提交, 当堂出分。

课后任务与拓展

- 必做① 读清单: 打开 [github.com/modelcontextprotocol](https://github.com/modelcontextprotocol/servers) 的 servers 仓库, 挑 3 个参考 Server, 各列出它声明的 2 个 Tool (名字 + 一句话作用), 并标出哪些有副作用 (200 字以内)。
- 必做② 画链路: 以“值班助手回答三号反应器温升问题”为例, 画出八步链路图, 标明每一步由 Host / Client / Server / LLM 中的谁完成。
- 选做③ 动手接: 在 [阿里云百炼控制台](#) (参考 [百炼文档 MCP 章节](#)) 给一个智能体接入一个现成 MCP Server, 截图记录“它自动发现了哪些工具”。

拓展阅读: MCP 规范原文与各语言 SDK ([GitHub 官方仓库](#)); [DeepSeek API 文档](#) Function Calling 章节 (温习第③步)。

下讲预告: AI Agent ——从工具调用到智能体时代

有了手, 谁来指挥

MCP 给了模型一排工具, 下一步是让它自己决定先用哪个、再用哪个

感知 → 规划 → 行动 → 反思

Agent 的一个循环, 和第三部分的“再循环一轮”是同一件事

什么时候会跑偏

多步调用后错误会累积, 为什么要设步数上限和人工检查点

化工里的 Agent

从“回答温升为什么偏快”到“自己去查、去算、去写处置建议”

多 Agent 协作

一个查数据、一个算工艺、一个写报告, 怎么分工

Q: 课前思考: 你愿意让 AI “自己决定” 做几步? 一步、三步, 还是直到做完为止? 把你的底线带来。

Thank
You!

 Your opinion
Matters

第六讲 · MCP ——让 AI 动起来的新“万能插座”
周天航 · 中国石油大学 (北京)

下课。请完成课后任务, 下次课把你“愿意让 AI 自己做几步”的底线带来。